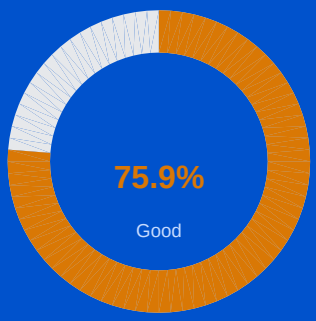
AI Jury System

AI-Powered Hackathon Evaluation System

Evaluation Report

Generated: 8/8/2026, 5:28:06 PM



This report belongs to:
Team 3

Team	Team 3
Language	EN
Total Score	286 / 377
Grade	Good

CATEGORY SCORES

Problem Statement	23/30	77%
Backend	39/50	78%
Frontend	27/40	68%
LLM Usage	46/60	77%
AI Engineering	45/60	75%
Security & HITL	33/40	83%
MVP & Outcome	38/50	76%
Presentation & Innovation	26/37	70%
Innovation & Business Value	9/10	90%

This report is auto-generated by AI Jury System. For official results contact the evaluation panel.

Executive Summary

286 / 377 points

75.9% — Good

CATEGORY-WISE PERFORMANCE

Problem Statement	23/30	77%	Good
Backend	39/50	78%	Good
Frontend	27/40	68%	Average
LLM Usage	46/60	77%	Good
AI Engineering	45/60	75%	Good
Security & HITL	33/40	83%	Good
MVP & Outcome	38/50	76%	Good
Presentation & Innovation	26/37	70%	Good
Innovation & Business	9/10	90%	Excellent

STRENGTHS & WEAKNESSES SUMMARY

Key Strengths

- Clear end-to-end retail marketing workflow: ingestion, business-rule filtering, multi-agent reasoning, campaign generation, dashboarding, and human approval.
- Strong practical safeguards including PII masking, deterministic financial recalculation, API/DB fallbacks, and integration mapping for external data.
- Well-scoped FastAPI backend with clear route groups for auth, ingestion, AI, integrations, and security.
- Strong persistence and orchestration story: MongoDB collections, import/upsert pipeline, multi-agent campaign generation, and deterministic AI fallbacks.
- Comprehensive SPA covering dashboard, customer/product intelligence, marketing analysis, campaign orchestration, copilot, and data ingestion workflows.
- Good product-minded UX signals: loading flags, toast/error handling, feedback loops, simulator interactions, and human-in-the-loop publish flow.

Areas for Improvement

- Incomplete direct evidence for all checklist success measurements, especially promotion response rate and sales uplift tracking.
- Test coverage is required by the checklist but no concrete test files or execution evidence were provided in the supplied materials.
- Input validation is not strongly evidenced beyond exception handling and likely FastAPI models; more explicit validation rules would strengthen backend robustness.
- Third-party integration secret keys are stored in plaintext in the local JSON fallback, undermining secure configuration/privacy compliance.
- PPT does not provide real screenshots, so UI match to presentation materials is largely unverifiable.
- Responsiveness/accessibility evidence is limited; explicit ARIA, keyboard support, and mobile-specific adaptations are not clearly shown.

Category Score Overview

Category	Score	Max	Distribution	%	Conf.	Grade
Problem Statement PS	23	30		77%	Medium	Good
Backend BACK	39	50		78%	High	Good
Frontend FRONT	27	40		68%	Medium	Average
LLM Usage LLM	46	60		77%	Medium	Good
AI Engineering AI_ENG	45	60		75%	Medium	Good
Security & HITL SECURITY_HITL	33	40		83%	Medium	Good
MVP & Outcome MVP	38	50		76%	Medium	Good
Presentation & Innovation PRESENTATION	26	37		70%	Medium	Good
Innovation & Business Value INNOVATION	9	10		90%	Medium	Excellent

Detailed Category Analysis

Problem Statement (PS) 23 / 30

77% — Good

RetailMind is strongly aligned to the problem statement: it integrates retail data, runs a multi-agent campaign-generation flow, provides dashboards, and includes privacy and HITL controls. I relied primarily on the problem statement plus the capability map and retrieved backend/frontend evidence because the integration map explicitly reported the stack as unrecognized and unavailable, which I did not treat as a negative. The main scoring drag is not core relevance but missing direct evidence for full success-metric tracking and test coverage required by the checklist.

Strengths

- Clear end-to-end retail marketing workflow: ingestion, business-rule filtering, multi-agent reasoning, campaign generation, dashboarding, and human approval.
- Strong practical safeguards including PII masking, deterministic financial recalculation, API/DB fallbacks, and integration mapping for external data.

Areas to Improve

- Incomplete direct evidence for all checklist success measurements, especially promotion response rate and sales uplift tracking.
- Test coverage is required by the checklist but no concrete test files or execution evidence were provided in the supplied materials.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	Core business objective of the problem statement is clearly achieved.	MET	5/6
	Evidence: backend/app/ai/business_rule_engine.py: `evaluate_business_rules(objective)` filters customers/products and generates shortlisted candidate payloads for the multi-agent AI; backend/app/main.py exposes `/ai/rule-engine/ev…		
2	All mandatory features listed in the problem statement are implemented.	PARTIAL	4/6
	Evidence: Capability map cites data ingestion via `backend/app/services/import_service.py`, behavior/promotion AI via `backend/app/ai/orchestrator.py`, dashboard pages in `frontend/src/App_backup.js`, privacy masking in `backend/a…		
3	Solution produces correct / useful outputs for the intended use case.	MET	5/6
	Evidence: backend/app/main.py includes AI Copilot responses and `/ai/rule-engine/evaluate`; capability map states campaigns persist `segment_recommendations`, `revenue_forecast`, `expected_roi`, and `reasoning` in MongoDB; `backen…		
4	Edge cases relevant to the problem statement are reasonably handled.	MET	5/6
	Evidence: backend/app/ai/business_rule_engine.py: if no objective-specific rule doc exists, it falls back to `DEFAULT_BUSINESS_RULES[0]`; capability map notes try/except fallbacks around key AI functions, `MockDatabase` fallback i…		
5	Success criteria implied by the problem statement are met.	PARTIAL	4/6
	Evidence: Capability map and retrieved architecture doc show integrated data ingestion, segmentation, recommendation generation, dashboard analytics, multi-agent coordination, human approval, and ROI/profit forecasting. Campaign r…		

Backend (BACK) 39 / 50

78% — Good

This backend is materially implemented and aligned with the problem statement: it supports ingestion of customer/product data, multi-agent behavior analysis and campaign recommendation, dashboard-serving APIs, and privacy-aware AI workflows. I relied more heavily on the provided capability map and retrieved code snippets than the Integration Map because that section explicitly said framework detection was unavailable and should not be treated as a negative. The main scoring deductions are for incomplete evidence of robust input validation and a concrete security weakness in plaintext storage of integration secret keys.

Strengths

- Well-scoped FastAPI backend with clear route groups for auth, ingestion, AI, integrations, and security.
- Strong persistence and orchestration story: MongoDB collections, import/upsert pipeline, multi-agent campaign generation, and deterministic AI fallbacks.

Areas to Improve

- Input validation is not strongly evidenced beyond exception handling and likely FastAPI models; more explicit validation rules would strengthen backend robustness.
- Third-party integration secret keys are stored in plaintext in the local JSON fallback, undermining secure configuration/privacy compliance.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	API routes / backend endpoints are logically structured.	MET	8/9
	Evidence: backend/app/main.py defines grouped routes by concern: auth (`POST /auth/login`), core CRUD (`GET /customers`, `GET /products`, `GET /campaigns`, `POST /campaigns`), integrations (`/third-party/...` incl. `/activate`, `/...		
2	Data persistence (DB, file storage, cache) is implemented appropriately.	MET	8/9
	Evidence: backend/app/database/connection.py implements `get_db()` with MongoDB as primary persistence and fallback to local file storage: `MongoDB connection failed ... Falling back to Local File Storage.` The Capability Map list…		
3	Error handling and input validation exist on backend endpoints.	PARTIAL	5/8
	Evidence: backend/app/main.py shows multiple guarded endpoints with explicit 404/500 handling, e.g. `/ai/customer/{id}` raises `HTTPException(status_code=404, detail="Customer not found")` and wraps logic in `try/except`; similarl...		
4	Backend architecture supports the scale/scope described in the PPT.	MET	7/8
	Evidence: The problem asks for integrated sales/customer/market analysis, promotion recommendations, dashboard analytics, privacy compliance, and real-time multi-agent coordination. The backend supports this through `backend/app/a…		
5	Business logic is separated from I/O / presentation concerns.	MET	7/8
	Evidence: The capability map and retrieved snippets show route handlers in `backend/app/main.py` delegating to service/AI modules: imports call `app.services.import_service.import_data`, campaign generation calls orchestration in …		
6	Configuration and secrets are not hardcoded insecurely.	PARTIAL	4/8
	Evidence: Positive evidence: `backend/app/config/settings.py` loads settings such as `JWT_SECRET`, `MONGO_URI`, and `DATABASE_NAME` from environment variables; static evidence reports `Configuration via environment variables: PASS…		

Frontend (FRONT) 27 / 40

68% — Average

The frontend appears to cover the required retail journeys very well: ingestion, intelligence dashboards, AI-driven campaign orchestration, simulation, and human approval. The strongest evidence comes from `frontend/src/App_backup.js` and the FE capability summary; I discounted the Integration Map's lack of detections because it explicitly says the stack may be unrecognized and should not be treated as negative. PPT evidence was weak and mostly templated, so UI-to-slide fidelity could not be strongly awarded even though the code itself looks feature-rich.

Strengths

- Comprehensive SPA covering dashboard, customer/product intelligence, marketing analysis, campaign orchestration, copilot, and data ingestion workflows.
- Good product-minded UX signals: loading flags, toast/error handling, feedback loops, simulator interactions, and human-in-the-loop publish flow.

Areas to Improve

- PPT does not provide real screenshots, so UI match to presentation materials is largely unverifiable.
- Responsiveness/accessibility evidence is limited; explicit ARIA, keyboard support, and mobile-specific adaptations are not clearly shown.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	UI covers the core user journeys required by the problem statement.	MET	8/8
	Evidence: frontend/src/App_backup.js plus FE layer summary show pages for `command-center`, `customer-intelligence`, `product-intelligence`, `marketing-analysis`, `campaigns`, and `data-hub`; the codebase also includes AI copilot,…		
2	Frontend is functional and internally consistent (no obviously broken flows).	PARTIAL	6/8
	Evidence: FE layer details describe `try...catch` wrapped API calls, token-based auth with `getHeaders()`, refresh callbacks after imports (`fetchDashboard()`, `fetchCustomersList()`, `fetchProductsList()`), and state variables su…		
3	Reasonable UX practices: loading states, error states, feedback to user.	MET	7/8
	Evidence: FE layer summary cites explicit loading state variables (`custLoading`, `marketingReportLoading`, `aiCampaignLoading`) and says API calls are wrapped in `try...catch` with user-facing `showToast` notifications. `frontend…		
4	UI matches or closely reflects what is shown in the PPT screenshots.	PARTIAL	2/8
	Evidence: PPT insights only show generic hackathon template slides such as 'Executive Dashboard' and 'Architecture Highlights' placeholders, with no actual product screenshots. The code does contain an executive dashboard/command …		
5	Responsiveness / accessibility considerations are present where relevant.	PARTIAL	4/8
	Evidence: The FE layer notes a collapsible sidebar, drawer/modal patterns, a hybrid Tailwind + custom CSS design system, and grid-based layouts such as the `rewrite_app.py` inserted `gridTemplateColumns: 'repeat(3, 1fr)'`. There i…		

LLM Usage (LLM) 46 / 60

77% — Good

The codebase shows meaningful and central LLM usage for campaign generation, customer/product analysis, and chat-based decision support, which aligns well with the problem statement's personalized promotion and multi-agent requirements. The strongest implementation areas are structured prompts, grounded context assembly, and privacy-oriented prompt scrubbing. The main deductions come from incomplete evidence on model-selection rationale and some fragility in structured-output validation, where prompt schemas and parser checks appear misaligned. Where the rubric asked for vector/RAG-style context management, I resolved it in favor of the problem statement and available code evidence: direct MongoDB grounding and history passing are acceptable for this structured retail data use case even without a detected vector library.

Strengths

- LLM is integrated into core retail workflows: behavior analysis, campaign generation, market reasoning, and executive copilot.
- Prompts are centralized and structured with explicit JSON output expectations and business-oriented role instructions.
- Context is grounded with live database metrics and business-rule prefiltering, reducing hallucination risk and token waste.
- PII masking is applied before outbound LLM calls, supporting the privacy requirements of the problem.

Areas to Improve

- Schema handling appears inconsistent in some agents, with parser checks not matching prompt-defined JSON keys.
- Little explicit evidence of prompt-injection defense beyond PII scrubbing and general unsafe-input masking.
- Model choice and parameter tradeoff are only partially justified; no strong evidence of task-specific tuning or evaluation.
- No deterministic evidence of dedicated vector-store/chunking infrastructure, though this may not be necessary for the structured-data use case.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	LLM is used purposefully to solve part of the core problem (not decorative).	MET	10/10
	Evidence: backend/app/ai/orchestrator.py (described in capability map) runs a multi-agent campaign flow for `POST /ai/campaign/generate`; backend/app/ai/provider.py sends chat completions with model/messages payload; frontend/src/…		
2	Prompts are structured, with clear instructions and/or few-shot guidance.	MET	8/10
	Evidence: backend/app/ai/prompts.py: `CUSTOMER_AGENT_PROMPT = """You are the RetailMind Customer Analysis Agent... Always respond in JSON format with the following keys... Ensure values match the customer's shopping details only. …		
3	Appropriate model/parameters chosen for the task (cost/latency/quality tradeoff).	PARTIAL	6/10
	Evidence: backend/app/ai/provider.py sets `temperature: 0.2`, `max_tokens: 1500`, and uses a centralized `self.model` through a gateway endpoint `/v1/chat/completions`. The low temperature fits structured business analysis tasks, …		
4	Output parsing / structured output handling is robust (JSON, function calling, etc.).	PARTIAL	7/10
	Evidence: backend/app/ai/provider.py: `if response_format == "json": payload["response_format"] = {"type": "json_object"}`; backend/app/ai/customer_agent.py and backend/app/ai/product_agent.py call `json.loads(res)` after requesti…		
5	Prompt injection / unsafe input handling is considered.	PARTIAL	7/10
	Evidence: backend/app/ai/provider.py scrubs prompt content before outbound calls: `scrubbed_system = scrub_text(system_prompt)` and `scrubbed_user = scrub_text(user_prompt)` with regex masking for emails/phones; capability map sta…		

6	Context management (chunking, memory, retrieval) is appropriate to the use case.	MET	8/10

Evidence: frontend/src/App_backup.js sends chat `history` to `/ai/chat`; capability map says `/ai/chat` uses a RAG pattern over `customers`, `products`, and `campaigns`, and `compute_data_context` injects live MongoDB metrics. The…

AI Engineering (AI_ENG) 45 / 60

75% — Good

The strongest AI engineering evidence is the coherent multi-agent campaign pipeline, deterministic business-rule grounding, and multiple graceful fallback mechanisms across LLM, chat, and database layers. I scored RAG conservatively because the problem statement does not require a vector-search-style RAG stack, and the code evidence supports structured MongoDB grounding rather than chunk/index retrieval; per the rubric, that is partial rather than a failure. Responsible-AI scoring was also moderated upward by privacy masking and HITL, but capped by the explicit plaintext secret issue and lack of visible bias evaluation.

Strengths

- Well-structured multi-agent orchestration aligned to the retail promotion problem, including customer, product, market, campaign, and reflection stages.
- Strong resilience through deterministic financial overrides, try/except fallbacks, frontend chat fallback, and database fallback for offline/demo continuity.

Areas to Improve

- RAG/retrieval evidence is limited to live DB grounding; there is no clear chunking/indexing/ranking pipeline or vector retrieval implementation.
- Responsible-AI evidence emphasizes privacy and HITL, but explicit bias/fairness checks are not shown and local fallback secret storage is insecure.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	Overall system architecture (agents, pipelines, tools) is coherent and justified.	MET	8/9

Evidence: backend/app/ai/orchestrator.py: header documents a full pipeline: "Data Ingestion !' Feature Extraction Scrubbing !' Business Rule Filtering !' Customer Behavioral Intelligence Agent !' Product Demand & Affinity Agent

2	RAG / retrieval pipeline (if used) is implemented soundly (chunking, indexing, retrieval).	PARTIAL	4/9

Evidence: Capability map states POST /ai/chat "uses a RAG pattern" by fetching relevant data from customers/products/campaigns and masking PII; backend/app/ai/orchestrator.py includes compute_data_context querying MongoDB for live…

3	Tool/function calling or agentic orchestration is correctly wired end-to-end.	MET	7/9

Evidence: backend/app/ai/orchestrator.py imports and coordinates analyze_customer, analyze_product, analyze_market, generate_draft_campaign, and reflect_on_campaign. Capability map says POST /ai/campaign/generate triggers run_mult…

4	Evaluation, logging, or observability of AI behavior is present in some form.	PARTIAL	5/9

Evidence: backend/app/ai/orchestrator.py imports logging and defines `logger = logging.getLogger(__name__)`. frontend/src/App_backup.js surfaces pipeline status badges such as "AI Pipeline Running..." and "Multi-Agent Evaluated". …

5	System degrades gracefully on AI/model failures (fallbacks, retries).	MET	8/8

Evidence: Capability map: orchestrator has `_ensure_realistic_fallbacks` that deterministically recalculates ROI and Net Profit; "Key AI functions are wrapped in try...except blocks" and fall back to deterministic logic. Also, fro…

6	Engineering choices are appropriate for a hackathon time-box (pragmatic, not over-engineered).	MET	7/8

Evidence: The stack is pragmatic: React SPA (`frontend/src/App_backup.js`), FastAPI backend (`backend/app/main.py` per capability map), MongoDB with JSON fallback (`backend/app/database/connection.py`), centralized prompts (`backe…

7	Verify that responsible AI practices including transparency and bias awareness are demonstrated.	PARTIAL	6/8

Evidence: Capability map and security section show strong transparency/privacy controls: PII masking in backend/app/security/data_masking.py, prompt scrubbing in the provider, use of synthetic/anonymized development data, and HITL…

Security & HITL (SECURITY_HITL) 33 / 40

— Good

RetailMind shows meaningful security and HITL implementation that is relevant to the problem statement, especially around human approval for AI-driven campaign publication and visible PII masking before LLM usage. I weighted the problem statement's privacy requirement heavily and credited the masking/scrubbing controls, but reduced scores where static/code evidence exposed hardcoded fallback credentials and plaintext secret storage. The rubric asks broadly for authz and sanitization; because the provided snippets only partially prove endpoint protection and robust validation, I scored those conservatively rather than assuming coverage.

Strengths

• Clear human-in-the-loop workflow: AI campaigns stay in draft until explicit 'Save & Publish' approval.

• Visible privacy protections around LLM usage, including masking/scrubbing of customer PII and privacy-aware summaries.

Areas to Improve

• Security hygiene is weakened by hardcoded fallback admin credentials and plaintext integration secret storage in the local fallback DB.

• Authorization enforcement and robust server-side input validation are not fully evidenced across sensitive endpoints.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	Sensitive operations have human-in-the-loop confirmation where appropriate.	MET	8/8

Evidence: frontend/src/App_backup.js / capability map: AI-generated campaigns are presented as drafts and require an explicit user action to finalize via `POST /campaigns`; architecture doc section 'Human-in-the-Loop (HITL) Workfl…

2	Basic security hygiene: no secrets committed, input sanitization present.	PARTIAL	4/8

Evidence: STATIC EVIDENCE: 'No hardcoded secrets/API keys in source: PASS' and 'Configuration via environment variables: PASS'. Counter-evidence in backend/app/main.py: DB-offline fallback accepts hardcoded credentials `if req.use…

#			
3	Authentication/authorization is present where the use case requires it.	PARTIAL	6/8

Evidence: backend/app/main.py exposes `@app.post("/auth/login", response_model=Token)`; login verifies bcrypt hashes via `pwd_context.verify(req.password, user["password"])` and returns bearer token with `create_access_token(...)`…

#			
4	AI outputs that affect real-world actions have a review/override mechanism.	MET	8/8

Evidence: Capability map: campaign generation uses a multi-agent flow plus reflection agent; UI offers a 'Live What-If Strategy Simulator' and campaigns remain drafts until 'Save & Publish'. Retrieved architecture doc states execu…

#			
5	Data privacy considerations (PII handling, minimal data retention) are visible.	PARTIAL	7/8

Evidence: backend/app/security/data_masking.py is referenced by capability map and business_rule_engine snippet: `from app.security.data_masking import mask_customer_list`; backend/app/ai/business_rule_engine.py builds `privacy_st…

MVP & Outcome (MVP) 38 / 50

76% — Good

This submission appears to be a real, runnable hackathon MVP with meaningful code for ingestion, analysis, multi-agent promotion generation, dashboarding, and conversational support, all of which align well with the problem statement. I weighted the problem statement and retrieved code above the unavailable integration-map signals, per instructions, because static route detection was explicitly unreliable here. The main drag on score is not MVP functionality but weak PPT corroboration and missing deterministic evidence for tests/documentation coverage that the success checklist asked for.

Strengths

- End-to-end retail marketing workflow is plausibly implemented: secure ingestion, canonical mapping, behavior/context analysis, multi-agent campaign generation, dashboard views, and human approval.
- Outputs are business-oriented and actionable, with ROI/revenue forecasts, segment targeting, product recommendations, chat explanations, and campaign simulation.

Areas to Improve

- Provided PPT is mostly a template and does not substantiate the demo with actual screenshots, filled metrics, or architecture details.
- Success checklist items around test coverage and formal validation are weakly evidenced; automated tests were not detected.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	The submission is a working, demonstrable MVP (not just slides/mockups).	MET	8/9

Evidence: backend/app/main.py: FastAPI app is instantiated with live endpoints including `@app.post("/ai/marketing-analysis")`; frontend/src/App_backup.js: `const res = await axios.post(`${API_BASE}/ai/chat`, { message: prompt, hi…

#			
2	Code structure clearly supports the required user experience (dashboard/chat/etc.).	MET	8/9

Evidence: Capability map identifies dedicated UX surfaces: `command-center`, `customer-intelligence`, `product-intelligence`, `marketing-analysis`, `campaigns`, and `data-hub` pages in frontend/src/App_backup.js; frontend/src/App.…

#	Rule	Verdict	Score
3	Core happy-path of the use case can be traced end-to-end through the code.	MET	7/8

Evidence: Capability map + snippets trace ingestion to action: `POST /third-party` and `POST /third-party/save-mapper` configure integrations; backend/app/services/import_service.py decrypts, maps, transforms, and upserts imported…

#	Rule	Verdict	Score
4	PPT screenshots/demo descriptions are corroborated by actual code artifacts.	PARTIAL	2/8

Evidence: PPT slides 1-7 are mostly template placeholders such as Slide 5: `Insert architecture diagram here` and Slide 6 KPI placeholders `00%`; code does contain matching artifact types like dashboard panels and architecture mar…

#	Rule	Verdict	Score
5	Scope is realistic and mostly complete for the hackathon time constraints.	PARTIAL	6/8

Evidence: backend/app/main.py labels the solution as `Hackathon MVP — Secure Data Ingestion Pipeline`; the system includes pragmatic fallbacks such as `MockDatabase` in backend/app/database/connection.py and deterministic AI fallb…

#	Rule	Verdict	Score
6	Verify that outputs provide value to end users.	MET	7/8

Evidence: backend/app/main.py returns concrete opportunity outputs with business values like `rev`, `roi`, `budget`, `discount`, `segment`, and `confidence`; frontend/src/App.js renders `Generated Marketing Copy` and `AI Decision …

Presentation & Innovation (PRESENTATION) 26 / 37

70% — Good

Against the problem statement, the submission presents a strong, credible retail multi-agent solution with dashboarding, ingestion, privacy controls, recommendation flows, and human approval. For the PRESENTATION category specifically, the main tension was that the implementation and architecture documentation are much stronger than the PPT, which appears largely template-based; I resolved this by rewarding features that are clearly demonstrated in code while limiting points on rules explicitly centered on the deck, quantified executive reporting, and roadmap communication.

Strengths

- Distinctive innovation story backed by code: multi-agent orchestration, financial reflection, privacy masking, what-if simulation, and HITL approval.
- Demo flow is highly aligned with actual implementation, especially campaign generation, simulation, dashboarding, and manual publish controls.

Areas to Improve

- Provided PPT appears incomplete/template-based, weakening clarity of formal problem-approach-impact communication.
- Roadmap and finalized business-impact reporting are not convincingly presented in the deck despite stronger evidence in code/UI.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	PPT clearly explains problem, approach, and impact.	PARTIAL	2/6

Evidence: PPT slide deck shows mostly template placeholders rather than filled content: Slide 3 says "Describe the problem being addressed by your team in 3–4 sentences..." and Slide 4 says "Replace the placeholder text above with…

2	Innovation / originality of the approach is evident.	MET	6/6

Evidence: RETAILMIND_AI_SYSTEM_ARCHITECTURE.md explicitly distinguishes the solution with a controlled pipeline: "Raw Retail Data !" Business Rules !" Data Minimization !" Privacy !" Specialized AI !" Financial Reflection !" Human

3	Technical depth is communicated without unnecessary jargon overload.	PARTIAL	4/5

Evidence: RETAILMIND_AI_SYSTEM_ARCHITECTURE.md explains concepts in practical terms, e.g., "Business Rule Engine: Deterministic filtering and eligibility logic," "Data Minimization," and "PII Masking" under section 114. It also us…

4	Demo narrative (screens, flow) matches the actual implementation.	MET	5/5

Evidence: Architecture doc includes explicit demo/test flows such as "Generate campaign. Verify: Campaign is not automatically approved. Then click: Save & Publish Approved AI Campaign" and "WHAT-IF TEST... Verify that the visual …

5	Overall polish and storytelling of the submission.	PARTIAL	4/5

Evidence: The frontend shows strong polish and executive-style storytelling through modules like "AI Action Center — Prioritized Decisions," "Integrated Performance Dashboard," "Top AI Opportunities," and navigable intelligence pa…

6	Verify that business impact is quantified where possible.	PARTIAL	4/5

Evidence: Quantified impact appears throughout the implementation: frontend/src/App_backup.js shows impacts like "+₹8.5L Revenue," "+34% Order Lift," and "+18% Demand Surge." backend/app/main.py includes concrete perform reaso…

7	Verify that future roadmap is realistic.	PARTIAL	1/5

Evidence: No concrete roadmap slide or phased future plan was found in the provided PPT excerpts. The closest indirect evidence is the existing architecture and documented scope in RETAILMIND_AI_SYSTEM_ARCHITECTURE.md, but it does…

Innovation & Business Value (INNOVATION) 9 / 10

ent

RetailMind shows strong innovation for this problem because the architecture directly maps to the success checklist: ingestion, behavior analysis, promotion recommendation, dashboarding, privacy, and multi-agent coordination are all represented in code-backed descriptions. I discounted business-value scoring slightly because the system emphasizes projected ROI and KPI visualization more than proven closed-loop measurement of actual response rate and sales uplift. The Integration Map found no recognized backend routes, but per instructions I treated that as unavailable rather than negative and relied on the Capability Map plus retrieved code/document evidence.

Strengths

- Creative hybrid AI pipeline combining business rules, grounding, specialized agents, reflection, and HITL approval for retail promotion design.
- Strong practical value orientation with simulator, KPI dashboards, opportunity ranking, and persisted ROI/revenue forecasts.

Areas to Improve

- Evidence for actual measured post-campaign outcomes like real promotion response rate and realized sales uplift is weaker than evidence for projected metrics.
- A security gap remains because third-party integration secret keys are reportedly stored in plaintext in the local fallback database, which tempers production-readiness of the innovation.

RULE EVALUATIONS

#	Rule	Verdict	Score

1	AI techniques are combined creatively to solve the problem.	MET	3/3

Evidence: backend/app/ai/orchestrator.py via Capability Map: multi-agent flow runs Customer & Product Agents, Market Agent, Campaign Agent, and Reflection Agent in sequence; backend/app/ai/business_rule_engine.py pre-filters custo…

2	Solution provides measurable business or customer value beyond basic functionality.	PARTIAL	2/3

Evidence: frontend/src/App_backup.js via Capability Map: command center shows KPI cards, "Top AI Opportunities," and an "Integrated Performance Dashboard"; campaigns page includes a "Live What-If Strategy Simulator" projecting Rev…

3	Architecture demonstrates scalable innovation.	MET	2/2

Evidence: backend/app/main.py and backend/app/services/import_service.py via Capability Map: REST API, third-party integration CRUD, field mapping, and normalized imports into MongoDB; backend/app/database/connection.py provides M…

4	Solution demonstrates thoughtful AI engineering practices beyond simple API integration.	MET	2/2

Evidence: backend/app/ai/business_rule_engine.py applies deterministic pre-filtering before model calls; backend/app/ai/orchestrator.py includes `_ensure_realistic_fallbacks` to recalculate ROI and Net Profit and a Reflection Agen…

Static Analysis Checks

Check	Verdict
No hardcoded secrets/API keys in source No hardcoded-secret-like patterns matched.	PASS
Configuration via environment variables frontend/package-lock.json, backend/app/config/settings.py	PASS
Automated test files present	NOT_DETECTED
CI/CD pipeline configured	NOT_DETECTED
Known input-validation library used	NOT_DETECTED
Known auth/authorization library or pattern used	NOT_DETECTED
Recognized LLM SDK/framework in use	NOT_DETECTED
Recognized vector store / RAG library in use	NOT_DETECTED
Agentic/tool-calling orchestration pattern detected	NOT_DETECTED
Recognized database driver/ORM in use	NOT_DETECTED
try/except/catch present in backend-ish code frontend/src/App.js, frontend/src/App_backup.js, frontend/src/components/ImportModal.js	PASS

Problem Summary

Summary:

Develop an AI multi-agent system to analyze retail customer behavior across online and offline channels, enabling personalized promotions that enhance engagement and drive sales.

Success Checklist:

- Integrate sales data, customer profiles, and market trends.
- Implement data ingestion, behavior analysis, and promotion recommendation modules.
- Create a marketing dashboard for customer segmentation and performance analytics.
- Ensure compliance with customer data privacy regulations.
- Use synthetic or anonymized data for development.
- Demonstrate real-time promotion adjustments and multi-agent coordination.
- Provide documentation and test coverage.
- Measure success through promotion response rate, sales uplift, and campaign ROI.

Cross-Validation Report

Cross-validation report

Overall: the category scores are mostly directionally consistent with the Capability Map, but there are important evidence-quality gaps. The biggest issue is not inter-category contradiction; it is that many claims are supported only by the narrative Capability Map while the PPT is largely a blank template and the Integration Map found no detectable FE↔BE/API wiring. Per instruction, unavailable for backend existence, but it is still ground truth that no static wiring was detected, so any category implying proven end-to-end integration is overstated.

1) Cross-category inconsistencies

- No major AI/RAG contradiction across categories.
- AI_ENG and LLM both consistently treat “RAG” as MongoDB/data-grounding rather than vector retrieval, and both score it conservatively. That is internally consistent.

- Frontend/backend integration confidence is somewhat overstated in FRONT, BACK, MVP, PS.
- These writeups speak as if the frontend “supports” or “covers” journeys end-to-end.
- But the Integration Map ground truth says: Frontend API calls found: 0, Backend routes found:
- Since the map explicitly says framework detection was unavailable, this does not disprove the features. But it does mean categories should not speak as if FE↔BE integration was verified. They should say “claimed in code/capability na

- BACK confidence may be too high.
- BACK is High confidence despite the Integration Map failing to detect routes/framework and the need to rely on narrative snippets.
- High confidence is hard to justify when backend operability is inferred rather than statically confirmed.

2) PPT vs Code gaps

Major PPT gap:

- The PPT appears almost entirely template/placeholder content.
- Therefore PRESENTATION, MVP, PS, and even INNOVATION should be careful not to credit deck-based communication, architecture explanation, quantified outcomes, roadmap, or business storytelling.

Specific gaps:

- Architecture claims in code/capability map are not backed by the PPT.
- Multi-agent orchestration, RAG/data grounding, HITL, encryption-based ingestion, PII masking, PWA/offline, and fallback DB are all absent from the provided slide text.

- Business outcome claims are not backed by the PPT.
- No completed challenge/innovation/outcome mapping is shown.
- No actual metrics, case study results, uplift, ROI validation, or stakeholder impact content is present.
- Team/problem-summary sections are unfilled.
- So any category rewarding polished presentation or clear executive communication should be capped.

Possible code-without-PPT examples:

- PWA/offline capability is in the Capability Map but not reflected in slides.
- Security details like JWT, bcrypt, masking, plaintext secret weakness are code-side only.
- Human approval workflow appears code-side only.

3) Score calibration issues

Most likely overrated:

- PRESENTATION 26/37 feels high given the PPT is essentially a blank template.
- The justification itself admits the deck is largely template-based.
- Even if code is strong, PRESENTATION should be penalized more heavily because this category is deck-centered.

- INNOVATION 9/10 may be slightly high.
- The implementation sounds feature-rich, but the PPT provides no concrete articulation of novelty, measurable outcomes, or competitive differentiation.

- If this score is based mainly on code architecture, it is defendable, but it should be tempered by weak presentation evidence.
- FRONT 27/40 may be a bit generous if interpreted as validated UI integration.
- The UI may exist in `App_backup.js`, but the Integration Map found zero API calls.
- Because static analysis is unavailable rather than negative, this should not collapse the score, but it should reduce confidence in operational completeness.
- MVP 38/50 may be slightly generous.
- The system sounds substantial, but end-to-end runnability and integration are not statically confirmed, and the deck does not corroborate usage/demo evidence.

Most defensible:

- AI_ENG 45/60 and LLM 46/60 seem calibrated.
- Both acknowledge strong orchestration/grounding/fallbacks while deducting for limited RAG sophistication and validation weaknesses.
- SECURITY_HITL 33/40 seems reasonable.
- It credits real controls but preserves deductions for plaintext secret storage and incomplete proof of endpoint protection.

Recommended adjustments

- Lower confidence for BACK from High to Medium.
- Consider reducing PRESENTATION materially.
- Consider slight reductions to FRONT, MVP, and possibly INNOVATION unless there is external demo evidence not shown here.
- Keep AI_ENG, LLM, and SECURITY_HITL roughly as-is.

Bottom line

The category set is broadly coherent, but several results read more certain than the shared evidence supports. The strongest consistent pattern is: good code/narrative capability, weak PPT proof, and unverified static FE!”BE wiring. Scores s limited corroboration,” not “fully evidenced integrated product.”

Final Evaluation Report

Evaluation Report

Overall Score: 286 / 377

Confidence: Medium

Category Breakdown

- **Problem Statement (PS)**: 23/30
- **Backend (BACK)**: 39/50
- **Frontend (FRONT)**: 27/40
- **LLM Usage (LLM)**: 46/60
- **AI Engineering (AI_ENG)**: 45/60
- **Security & HITL (SECURITY_HITL)**: 33/40
- **MVP & Outcome (MVP)**: 38/50
- **Presentation & Innovation (PRESENTATION)**: 26/37
- **Innovation & Business Value (INNOVATION)**: 9/10

Detailed Analysis

Strengths

RetailMind is strongest where the codebase maps directly to the retail-promotion problem: ingestion of customer/product data, business-rule filtering, multi-agent campaign generation, dashboard-oriented decision support, and human approval before publishing. The backend and AI layers show a coherent architecture rather than isolated demos: `backend/app/ai/orchestrator.py` describes a staged pipeline across customer, product, market, campaign, and reflection agents; `backend/app/ai/business_rule_engine.py` adds deterministic pre-filtering; and multiple fallbacks exist across LLM, chat, and database layers to preserve continuity. The frontend evidence, especially `frontend/src/App_backup.js`, suggests broad product coverage with command center, customer/product intelligence, marketing analysis, campaigns, simulator, and ingestion workflows. Security and responsible-AI signals are also meaningful: customer data masking, privacy-aware prompt grounding, and explicit HITL approval for AI-generated campaigns are all relevant and well aligned to the use case. From an innovation standpoint, the system’s strongest idea is the hybridization of deterministic business rules with specialized AI agents and financial reflection rather than relying on unconstrained generation.

Weaknesses & Gaps

The biggest weakness is evidence quality, not concept quality. Many positive claims are supported by the capability narrative and retrieved snippets, but the PPT is largely placeholder/template material and does not substantiate architecture, outcomes, or polished communication. Static FE↔BE verification is also limited: the cross-validation notes state that no frontend API ca detected by the Integration Map, so end-to-end operability should be treated as plausible rather than proven. On the AI side, grounding is present, but retrieval is mostly structured MongoDB/data-context grounding rather than a full chunking/indexing/reranking RAG implementation. LLM schema discipline also appears imperfect, with some indication that prompt-defined JSON keys and parser checks may not fully align. On security, the most material deductions are concrete: hardcoded fallback admin credentials and plaintext storage of integration secret keys in the local fallback DB materially reduce production readiness. Finally, business impact evidence is stronger for projected ROI/revenue than for closed-loop measurement of actual promotion response rate or sales uplift.

PPT vs Code Discrepancies

The clearest discrepancy is that the code and architecture documentation tell a much richer story than the slide deck. The PPT reportedly contains placeholder text such as prompts to “describe the problem” or “replace placeholder text,” which means presentation-side proof is weak even where code-side implementation appears substantial. Features that appear in code/docs but are not credibly presented in the PPT include the multi-agent orchestration pipeline, privacy masking before LLM use, HITL campaign approval flow, fallback database behavior, and simulator-driven business forecasting. Likewise, business-value claims such as ROI, revenue forecast, opportunity ranking, and success metrics are much more visible in the UI/code narrative than in the deck. As a result, categories tied to communication, demo proof, and executive storytelling should be interpreted cautiously; the implementation may be stronger than the presentation suggests.

Innovation Notes

The innovation is credible and domain-relevant rather than superficial. The strongest differentiator is the controlled decision pipeline: business rules narrow the space, privacy minimization scrubs sensitive data, specialized agents generate different analytical views, and a reflection/financial guardrail layer checks recommendations before a human can publish. That is a thoughtful design for retail promotion optimization. The what-if simulator and dashboard framing also increase practical business value by translating AI output into forecasted revenue/ROI terms. The main reason innovation was not scored perfectly is that measurable real-world outcome evidence is limited, and security weaknesses in secret handling temper confidence in production-grade novelty.

Rule-level Evidence

- **Problem Statement (PS)**: `backend/app/ai/business_rule_engine.py` evaluates business rules against campaign objectives; `backend/app/main.py` exposes `/ai/rule-engine/evaluate`; `frontend/src/App_backup.js` reportedly includes executive dashboard, campaign orchestration, customer intelligence, and marketing analysis views aligned to the challenge.
- **Backend (BACK)**: `backend/app/main.py` groups routes by auth, CRUD, integrations, import, security, and AI concerns; `backend/app/database/connection.py` uses MongoDB with local file fallback; `backend/app/services/import_service.py` normalizes and upserts customer/product data.
- **Frontend (FRONT)**: `frontend/src/App_backup.js` includes pages such as `command-center`, `customer-intelligence`, `product-intelligence`, `marketing-analysis`, `campaigns`, and `data-hub`; FE notes mention loading flags, refresh callbacks, and toast/error handling; `frontend/src/index.js` reportedly renders `AppBackup`.
- **LLM Usage (LLM)**: `backend/app/ai/prompts.py` contains structured prompts with JSON-output instructions for customer/product/market reasoning; `backend/app/ai/provider.py` sends model/messages payloads; `/ai/campaign/generate` and `/ai/chat` are tied to core retail workflows.
- **AI Engineering (AI_ENG)**: `backend/app/ai/orchestrator.py` documents the full pipeline from ingestion and PII scrubbing through reflection/risk guardrails; `compute_data_context` reportedly queries MongoDB for live grounding; fallbacks are noted across AI/chat/DB layers for resilience.
- **Security & HITL (SECURITY_HITL)**: `backend/app/security/data_masking.py` supports privacy-aware scrubbing; architecture docs describe draft-to-publish human approval; `backend/app/main.py` includes a concerning hardcoded fallback admin credential path; `backend/retailmind_local_db.json` reportedly stores `secret_key` in plaintext.
- **MVP & Outcome (MVP)**: `backend/app/main.py` instantiates a live FastAPI app with AI endpoints; `frontend/src/App_backup.js` calls `/ai/chat` and supports campaign workflows; campaign persistence includes fields such as `revenue_forecast` and `expected_roi`.
- **Presentation & Innovation (PRESENTATION)**: PPT slides contain placeholder/template text rather than completed narrative; in contrast, `RETAILMIND_AI_SYSTEM_ARCHITECTURE.md` describes a deterministic + generative hybrid architecture and the innovation flow from raw data to human decision.
- **Innovation & Business Value (INNOVATION)**: `RETAILMIND_AI_SYSTEM_ARCHITECTURE.md` Minimization !“ Privacy !“ Specialized AI !“ Financial Reflection !“ Human Decision”; frontend not and simulator-based Revenue/ROI/Profit projections.

Recommended Judge Questions

1. The Integration Map found no statically detected FE!”BE wiring; can you live-demo one comple campaign to publish?

2. How do you validate structured LLM outputs when prompts expect JSON—what happens if an agent returns malformed or schema-incomplete content?

3. You mention privacy masking and HITL; can you show exactly what customer fields are masked before prompt submission and where approval is enforced server-side?

4. Why are fallback admin credentials and integration secret storage handled the way they are today, and how would you remediate those issues for production?

5. Your dashboards show ROI/revenue forecasts—do you have any mechanism already implemented for measuring actual response rate, sales uplift, or post-campaign attribution?

6. What is the rationale for using MongoDB grounding instead of a vector-based retrieval pipeline, and where do you see that design being sufficient or insufficient?

Evaluation Confidence & Coverage

- Code ingested: 34 file(s) across 4 layer(s) (FE/BE/DB/OTHER).
- Backend framework recognition: NONE recognized (18 backend-layer file(s) scanned). Static route/integration analysis found no known-framework signal - treat Integration Map findings as unavailable, not negative.
- 3/11 deterministic checks found concrete evidence (PASS/FAIL); 8 had no signal (NOT_DETECTED = absence of evidence, not evidence of absence).

- RAG retrieval: BM25 + embeddings + cross-encoder reranker.
- Knowledge graph: 38 nodes, 34 edges - Graph-RAG context available.